
PROBABILISTIC FORECASTING VIA AUTOREGRESSIVE FLOW MATCHING

Anonymous authors
Anonymous submission

ABSTRACT

We introduce FLOWTIME, a generative model devised for the probabilistic forecasting of multivariate timeseries data. Conditioning on historical measurements together with, where available, prospective covariates, we cast forecasting as the act of sampling from a learned conditional distribution defined over future trajectories. More precisely, we decompose the joint distribution over future observations into an ordered sequence of conditional densities, each of which is instantiated through a shared flow that transports a tractable base distribution onto the distribution of the subsequent observation, conditioned throughout on the observed covariates. This construction is rendered feasible by our appeal to the flow matching (FM) framework, which affords scalable and simulation-free estimation of the requisite transformations. Through the coupling of this factorization with the FM objective, FLOWTIME inherits the characteristic virtues of autoregressive models—namely, strong extrapolative behaviour, parsimonious model size, and well-calibrated uncertainty estimates—while concurrently accommodating the intricate, multi-modal conditional distributions that typify contemporary transport-based generative models. We substantiate the efficacy of FLOWTIME across a range of dynamical systems as well as real-world forecasting tasks.

1 INTRODUCTION

Among the foundational problems confronting modern machine learning is that of probabilistic timeseries forecasting, wherein the objective is to extrapolate, from observational data alone, the manner in which the dynamics of a system evolve into the future. The pervasiveness of this problem across scientific, industrial, and societal disciplines renders it a matter of enduring and cross-cutting significance (Lim & Zohren, 2021; Dama & Sinoquet, 2021; Ye et al., 2024).

An increasingly prominent line of inquiry seeks to marshal deep generative models toward this end (Karl et al., 2016; Rasul et al., 2020; Desai et al., 2021). Under this framing, the act of forecasting is construed as sampling from a prospective probability density that is conditioned upon the past. Of these, diffusion and score-based generative models (Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2020) have, of late, advanced the state of the art across a multiplicity of forecasting benchmarks (Rasul et al., 2021; Tashiro et al., 2021; Kollovich et al., 2023; Meijer & Chen, 2024). Yet, notwithstanding their considerable empirical prowess, diffusion models are ordinarily encumbered by substantial computational expenditure during both training and inference.

Flow Matching (FM) (Liu et al., 2022; Albergo et al., 2023; Lipman et al., 2022) constitutes an emergent paradigm for generative modeling that at once generalizes and subsumes diffusion models, all the while conferring a markedly greater latitude of design. In contradistinction to diffusion models, which are predicated upon iterative stochastic denoising steps enacted over a discretized trajectory, FM learns deterministic probability paths that transport arbitrary base distributions directly onto the target distribution by means of continuous normalizing flows. Through a judicious design of these probability paths, FM obviates both the handcrafted noise schedules and the protracted ancestral sampling chains endemic to diffusion, thereby engendering more efficient training and sampling.

More recently still, FM has been brought to bear upon timeseries modeling and probabilistic forecasting (Tamir et al., 2024; Hu et al., 2024; Kollovich et al., 2024), evincing compelling empirical results together with an improved computational economy relative to diffusion models. Prevailing

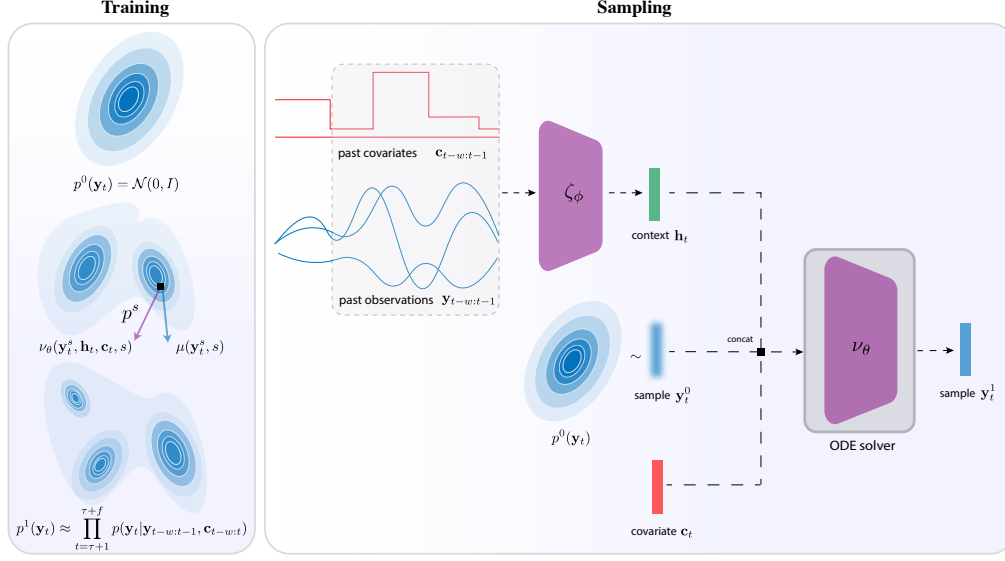


Figure 1: A synoptic overview of the training and sampling procedures that constitute our approach. Over the course of training, a probability path $(p^s)_{0 \leq s \leq 1}$ is constructed so as to interpolate between a base distribution p^0 and the target distribution p^1 , this path being generated by a velocity field μ . Training proceeds by regressing μ through a neural network ν_θ that ingests a sample from the probability path y_t^s at flow step s , a context vector h_t that encodes antecedent observations and covariates, and the prevailing covariate c_t . Once training has concluded, sampling from the target distribution is accomplished by first drawing from the base distribution p^0 and thereafter integrating the trained velocity field by means of an ODE solver until $s = 1$.

approaches, however, are contingent upon directly learning the conditional distribution of an entire fixed-horizon future window given a fixed-horizon context window. Although this expedites both training and sampling, it engenders models that extrapolate poorly beyond the support of the training distribution and that furnish miscalibrated uncertainty estimates. Compounding these difficulties, it precipitates a more onerous optimization problem, in which models must contend with the estimation of intricate, time-dependent probability paths along the flow dimension. This latter difficulty is brought into relief by (Kollovieh et al., 2024), whose authors advocate the use of informed, Gaussian-process-based priors so as to simplify and accelerate optimization—an expedient that, regrettably, confines their approach to univariate forecasting problems.

In the present work, we advance FLOWTIME, a deliberately parsimonious alternative that harnesses FM to learn probabilistic forecasting models scaling gracefully to high-dimensional multivariate datasets without recourse to informed priors. Whereas extant FM-based methods model the entire future window in a single, simultaneous stroke, FLOWTIME decomposes the forecasting problem into a sequence of conditional distributions, thereby modeling the distribution of each future time step conditioned upon antecedent observations and covariates. This decomposition is effected through a shared flow that transports a tractable base distribution onto the distribution of the ensuing observation. In consequence of this autoregressive structure, FLOWTIME accommodates variable forecasting horizons with ease, extrapolates beyond the training distribution, and yields well-calibrated uncertainty estimates, all the while attenuating the difficulty of the underlying optimization problem.

We demonstrate that FLOWTIME attains strong performance alike on classical dynamical systems and on real-world forecasting tasks, thereby furnishing a pragmatic and effective solution to the problem of probabilistic timeseries forecasting.

2 BACKGROUND: FLOW MATCHING

Flow matching (FM) (Lipman et al., 2022) is a paradigm for generative modeling that renders tractable the training of continuous normalizing flows (CNFs) (Chen et al., 2018; Grathwohl et al.,

2018) at a hitherto unprecedented scale. CNFs conceive of data transformations as the solutions to ordinary differential equations (ODEs), thereby furnishing invertible mappings endowed with tractable likelihoods.

In common with CNFs, FM sets out to learn a time-dependent diffeomorphic map defined upon the data space Ω —termed a flow $\psi: [0, 1] \times \Omega \rightarrow \Omega$. This flow transports a sample $\mathcal{X}^0 \sim p$ drawn from a source distribution p into a target sample $\mathcal{X}^1 := \psi(\mathcal{X}^0, 1)$, in such a manner that $\mathcal{X}^1 \sim q$ for some target distribution q . The flow is, in turn, constructed by solving the following initial value problem:

$$d\psi(\mathcal{X}, s) = \mu(\psi(\mathcal{X}, s), s)ds, \quad \psi(\mathcal{X}, 0) = \mathcal{X}, \quad s \in [0, 1] \quad (1)$$

where $\mu: [0, 1] \times \Omega \rightarrow \Omega$ denotes a vector field that specifies the velocity of the flow and, in so doing, generates a probability path $(p^s)^{0 \leq s \leq 1}$ in which each p^s is a distribution over Ω satisfying $p^0 = p$. The objective, accordingly, is to learn an admissible vector field μ for which $p^1 = q$.

In contradistinction to CNFs, wherein the vector field is learned through likelihood maximization—a procedure that necessitates solving and differentiating through Eq. (1) during training—FM instead learns a vector field ν_θ parameterized by θ by regressing against the vector fields of fixed conditional probability paths. This stratagem permits the specification of a tractable objective function, known as conditional flow matching (CFM), defined as:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{s \sim \mathcal{U}[0,1], \mathcal{X}^1 \sim q(\mathcal{X}^1), \mathcal{X} \sim p^s(\mathcal{X}|\mathcal{X}^1)} \|\nu_\theta(\mathcal{X}, s) - \mu(\mathcal{X}, s|\mathcal{X}^1)\|^2. \quad (2)$$

Minimization of this objective renders it possible to sample from q by solving Eq. (1) with the trained ν_θ . In dispensing with gradient backpropagation through the ODE solver, FM affords stable and scalable training for high-dimensional generative tasks, among them image and video modeling (Esser et al., 2024; Polyak et al., 2024). A diversity of probability-path designs and conditioning strategies has since been proposed, together serving to unify a plurality of transport-based generative models beneath this common framework (Liu et al., 2022; Albergo et al., 2023; Tong et al., 2023).

3 AUTOREGRESSIVE FLOW MATCHING

Problem Setting Let $\mathbf{y}_\tau \in \mathbb{R}^n$ denote an n -dimensional observation registered at time τ . Granted access to its l antecedent measurements, the current observation among them, denoted $\mathbf{Y}_l = \{\mathbf{y}_{\tau-l}, \dots, \mathbf{y}_\tau\}$, our aim is to render probabilistic forecasts of the ensuing f future values, denoted $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$, conditioned, where applicable, upon observed covariates $\mathbf{C} = \{\mathbf{c}_{\tau-l}, \dots, \mathbf{c}_{\tau+f}\}$ with $\mathbf{c}_\tau \in \mathbb{R}^c$. Stated formally, our objective is to learn to sample from the conditional distribution $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$ by recourse to a dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$, wherein m designates the number of instances.

We propose to factorize this conditional distribution autoregressively across future time-steps, invoking a Markov assumption of order w , whereby:

$$p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C}) = \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \quad (3)$$

where $w \leq l$ denotes the history size. Our objective thereby reduces to learning the conditional distribution set out in Eq. (3). In lieu of the maximum likelihood estimation customarily adopted for autoregressive models, we enlist the flow matching framework (Lipman et al., 2022) for the estimation of this conditional distribution.

This autoregressive factorization confers several attendant advantages. In the first place, it reduces the learning problem to the modeling of comparatively tractable conditional distributions $p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$, in preference to an unwieldy, high-dimensional joint distribution. Secondly, the conditional distribution associated with each time step admits training within a teacher-forcing framework, thereby permitting parallelization. Thirdly, the Markov assumption, parameterized by the window size w , negotiates a principled trade-off between model capacity and computational complexity. Finally, this formulation lends itself to interpretable uncertainty quantification, inasmuch as it explicitly models the manner in which predictive uncertainty propagates through time along the chain of conditional distributions.

3.1 CONSTRUCTING A TARGET PROBABILITY PATH

For every future time step $t \in [\tau + 1, \tau + f]$, we posit a target probability path $(p^s(\mathbf{y}_t))_{0 \leq s \leq 1}$ whose marginals satisfy $p^0(\mathbf{y}_t) = \mathcal{N}(0, I)$ and $p^1(\mathbf{y}_t) \approx \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t | \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \approx p_{\mathcal{D}}$, wherein $p_{\mathcal{D}}$ denotes the empirical distribution. Such a probability path is generated by a flow field $\psi: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ along s , itself defined through the ordinary differential equation (ODE):

$$d\psi(\mathbf{y}_t, s) = \mu(\psi(\mathbf{y}_t, s), s)ds, \quad \psi(\mathbf{y}_t, 0) = \mathbf{y}_t \quad s \in [0, 1] \quad (4)$$

where $\mu: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ is a vector field that constructs the flow, elected so as to ensure that the resultant flow satisfies the continuity equation.

With a view to deriving a tractable objective, we may model the marginal probability path as a mixture of conditional probability paths:

$$p^s(\mathbf{y}_t) = \int p^s(\mathbf{y}_t | \mathbf{z}) p(\mathbf{z}) d\mathbf{z} \quad (5)$$

where $\mathbf{z} \sim \pi_{0,1}(\mathbf{z})$ constitutes a conditioning random variable drawn from an arbitrary data coupling $\pi_{0,1}$. Theorem 2 of (Lipman et al., 2022) establishes that the conditional velocity fields $\mu(\mathbf{y}_t, s | \mathbf{z})$ associated with the conditional probability paths $p^s(\mathbf{y}_t | \mathbf{z})$ coincide with their respective marginal velocity field $\mu(\mathbf{y}_t, s)$.

In accordance with (Tong et al., 2023), we elect $\mathbf{z} = \{\mathbf{y}_t^0, \mathbf{y}_t^1\}$ with $p(\mathbf{z} = (\mathbf{y}_t^0, \mathbf{y}_t^1)) = p^0 \times p^1$, where $\mathbf{y}_t^0$ and $\mathbf{y}_t^1$ are samples drawn respectively from p^0 and p^1 . We then define the conditional probability path as:

$$p^s(\mathbf{y}_t | \mathbf{z}) = \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I) \quad (6)$$

whose marginals satisfy $p^0(\mathbf{y}_t | \mathbf{z}) = p^0(\mathbf{y}_t)$ and $p^1(\mathbf{y}_t | \mathbf{z}) = p^1(\mathbf{y}_t)$ in the limit $\sigma^2 \rightarrow 0$. The corresponding conditional velocity field is then, quite simply, $\mu(\mathbf{y}_t, s | \mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$. This choice yields rectilinear probability paths connecting the source and target distributions within Euclidean space, together with closed-form solutions, and thereby confers efficiency upon both training and sampling. Estimation of this velocity field permits us to sample from our target distribution $p(\mathbf{y}_t | \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$ by integrating the ODE of Eq. (4) until $s = 1$.

3.2 TRAINING

In keeping with the standard FM setting, we estimate this velocity field by regressing it against a neural network ν_θ with parameters θ under the conditional flow matching objective of Eq. 2. To accomplish this efficiently within our setting, we introduce a context vector $\mathbf{h}_t \in \mathbb{R}^h$ that encodes the context window at time t , defined by:

$$\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1}) \quad (7)$$

where $\zeta: \mathbb{R}^{w \times n} \times \mathbb{R}^{w \times c} \rightarrow \mathbb{R}^h$ denotes a neural network parameterized by ϕ . We may then articulate our learning objective as:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{y}_t^s \sim p^s(\mathbf{y}_t | \mathbf{z})} \|\mu(\mathbf{y}_t^s, s | \mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2 \quad (8)$$

which we optimize by randomly drawing an observation at time point t , together with its associated context window and covariates, from $\mathcal{D}$, and by jointly optimizing the parameters ϕ and θ through the minimization of $\mathcal{L}(\theta, \phi)$ via stochastic gradient descent. It bears remarking that this procedure is tantamount to teacher forcing (Williams & Zipser, 1989), in which ground-truth past observations—rather than the model’s own prior predictions—are supplied to the model during training, thereby permitting parallel training across all time steps whilst forestalling the accumulation of error. Algorithm 1 recapitulates the training procedure for autoregressive flow matching.

3.3 INFERENCE

Once training is complete, we seek to furnish probabilistic predictions, extending f timestamps into the future, for hitherto unseen observations $\mathbf{Y}_l$ and their corresponding covariates $\mathbf{C}$. To this end, we adopt the sampling procedure delineated in Algorithm 2, in which predictions are generated autoregressively by means of a rolling-window scheme. This procedure may be reiterated an arbitrary number of times so as to obtain empirical quantiles characterizing the uncertainty of our predictions.

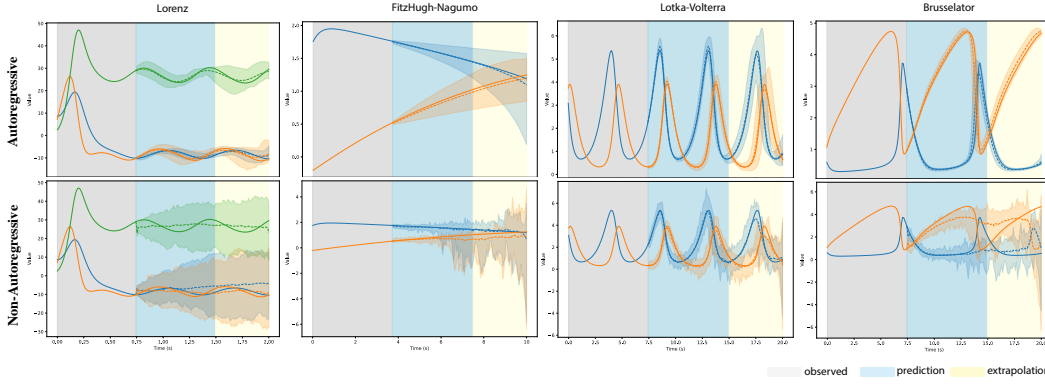


Figure 2: Representative samples drawn from the forecasting results of autoregressive flow matching set against standard flow matching, across four distinct dynamical systems. Solid lines denote the ground truth, whereas dashed lines trace the mean prediction together with its accompanying 95% confidence interval. The results are rendered under both the prediction and the extrapolation regimes.

Algorithm 1 Training: Autoregressive Flow Matching.

Require: Dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$, window size w , networks ν_θ, ζ_ϕ

- 1: **while** not converged **do**
- 2: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$ and $(\mathbf{y}_t^1, \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \sim \mathcal{D}$
- 3: Set $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$
- 4: Sample $s \sim \mathcal{U}(0, 1)$ and $\mathbf{y}_t^s \sim \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I)$
- 5: Compute target: $\mu(\mathbf{y}_t^s, s | \mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$
- 6: Compute loss: $\mathcal{L}_t = \|\mu(\mathbf{y}_t^s, s | \mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2$
- 7: Update θ, ϕ
- 8: **end while**
- 9: **return** ν_θ, ζ_ϕ

4 EXPERIMENTS

In this section, we set forth our empirical findings and juxtapose FLOWTIME against a range of baselines, drawing upon both real-world datasets and data procured through the simulation of assorted dynamical systems.

4.1 DYNAMICAL SYSTEMS

Data We appraised FLOWTIME on the task of forecasting five distinct stochastic dynamical systems: stochastic renderings of the Lorenz system (Lorenz, 1963), the FitzHugh–Nagumo model (FitzHugh, 1961), the Lotka–Volterra system Lotka (1925), the Brusselator (Lefever & Prigogine, 1968), and the Van der Pol oscillator (van der Pol, 1926). For each such system, we generated trajectories by numerically integrating the corresponding stochastic differential equations,

Algorithm 2 Sampling: Autoregressive Flow Matching.

Require: Past observations $\mathbf{Y}_l$, covariates $\mathbf{C}$, forecast horizon f , window size w , networks ν_θ, ζ_ϕ

- 1: Initialize $\mathbf{Y}_f \leftarrow \mathbf{Y}_l$
- 2: **for** $t = \tau + 1$ **to** $\tau + f$ **do**
- 3: Set context: $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$, using the last w entries in $\mathbf{Y}_f$
- 4: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$
- 5: Solve $\frac{d\psi(\mathbf{y}_t, s)}{ds} = \nu_\theta(\psi(\mathbf{y}_t, s), \mathbf{h}_t, \mathbf{c}_t, s), \quad \psi(\mathbf{y}_t, 0) = \mathbf{y}_t^0, \quad s \in [0, 1]$
- 6: Append $\psi(\mathbf{y}_t, 1)$ to $\mathbf{Y}_f$
- 7: **end for**
- 8: **return** $\mathbf{Y}_f$

employing randomly sampled initial conditions in concert with realizations of Brownian motion. The particulars of the dynamical systems, together with the data-generation process, are consigned to Appendix A.1.

Implementation details For these experiments, we instantiated ζ_ϕ as a two-layer bi-directional LSTM endowed with 64 hidden units, and ν_θ as an MLP comprising three hidden layers, each of size 64. We employed Fourier positional embeddings to encode the flow step s into a 16-dimensional vector by means of a fixed set of frequencies, and set the context length w equal to the prediction length. Our models were trained via stochastic gradient descent, using an Adam optimizer with a learning rate of 0.003 and a batch size of 128. Training to convergence required approximately 7 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Baselines In order to gauge the efficacy of the autoregressive factorization at the heart of FLOWTIME, we contrasted our approach with a non-autoregressive flow-matching baseline that models directly the joint distribution of future observations conditioned upon past observations and covariates, that is, $p^1 = p(\mathbf{Y}_f | \mathbf{Y}_l, \mathbf{C})$. This baseline functions as a direct ablation, serving to isolate the effect of the autoregressive factorization introduced in FLOWTIME. We observe, moreover, that this non-autoregressive formulation bears a close affinity to the method advanced in (Kollovieh et al., 2024). Further particulars concerning this baseline are consigned to Appendix B.

Evaluation In order to appraise the model’s capacity to extrapolate beyond the support of its training distribution, we partitioned each trajectory into three constituent segments: (1) an observation window, which furnishes the historical input to the model; (2) a prediction window, which is made available solely during training so as to serve as the target; and (3) an extrapolation window, which remains wholly unseen throughout training. We enlisted the continuous rank probability score (CRPS) (Winkler et al., 1996) to evaluate the model’s performance with respect to uncertainty quantification, and the normalized root mean square error (NRMSE) (Hyndman & Koehler, 2006) to appraise the accuracy of its point predictions.

Table 1: A comparison of performance between the autoregressive (FLOWTIME) and non-autoregressive (Non-AR) factorizations of our method, conducted across five distinct stochastic dynamical systems. Performance is adjudged in terms of CRPS and NRMSE under both the prediction and extrapolation regimes on the test set. The reported figures denote the mean and standard deviation computed across five distinct random seeds.

System	Factorization	Prediction		Extrapolation	
		CRPS ↓	NRMSE ↓	CRPS ↓	NRMSE ↓
Lorenz	Non-AR	0.147±0.012	0.242±0.018	0.120±0.009	0.301±0.024
	FLOWTIME	0.090±0.008	0.017±0.003	0.137±0.011	0.048±0.006
FitzHugh-Nagumo	Non-AR	0.165±0.014	0.131±0.011	0.214±0.016	0.306±0.022
	FLOWTIME	0.144±0.013	0.090±0.007	0.146±0.012	0.278±0.019
Lotka-Volterra	Non-AR	0.130±0.011	0.266±0.019	0.136±0.012	0.502±0.033
	FLOWTIME	0.126±0.010	0.189±0.015	0.123±0.009	0.339±0.026
Brusselator	Non-AR	0.217±0.016	0.658±0.042	0.238±0.019	1.046±0.057
	FLOWTIME	0.125±0.011	0.031±0.004	0.109±0.008	0.023±0.003
Van der Pol	Non-AR	0.143±0.012	0.161±0.014	0.211±0.018	0.510±0.031
	FLOWTIME	0.225±0.017	0.051±0.005	0.282±0.021	0.101±0.009

Results Our results, epitomized in Table 1, attest that the autoregressive factorization adopted in FLOWTIME markedly enhances forecasting performance across a multiplicity of stochastic dynamical systems. On the prediction task, the AR approach surpasses the non-AR baseline in terms of NRMSE across all five systems, the improvements being especially pronounced for the Lorenz (0.017 versus 0.242) and Brusselator (0.031 versus 0.658) systems, which correspond to reductions in error of 93% and 95%, respectively. The AR factorization further yields a superior CRPS in four

of the five systems, the Van der Pol oscillator constituting the sole exception, for which the non-AR approach achieves the better uncertainty quantification. Of greater consequence, these gains carry over into the extrapolation regime, wherein the autoregressive factorization sustains its advantage in NRMSE across every system, thereby evincing its superior capacity to generalize beyond the training distribution. This is nowhere more manifest than in the Brusselator system, for which the AR factorization curtails the extrapolation NRMSE by 98% (0.023 versus 1.046). Taken together, these findings corroborate that the modeling of temporal dependencies through autoregressive factorization enables FLOWTIME to apprehend more faithfully the underlying dynamics of complex stochastic systems, issuing in more accurate predictions and improved uncertainty quantification, particularly amid the exacting conditions of extrapolation.

4.2 REAL-WORLD DATASETS

Data We appraised FLOWTIME on the tasks of forecasting an assortment of univariate and multivariate timeseries of differing frequencies (hourly and daily) drawn from GluonTS (Alexandrov et al., 2020). In particular, we availed ourselves of the following datasets: Electricity (Lai et al., 2018), Solar (Lai et al., 2018), Exchange (Dua et al., 2017), Traffic (Dua et al., 2017), and Wikipedia (Gasthaus et al., 2019). Further particulars are furnished in Appendix A.2.

Implementation details We instantiated ζ_ϕ as a three-layer bi-directional LSTM endowed with 64 hidden units, and defined ν_θ through a residual neural network comprising five residual blocks, each realized by 1-D convolutional layers equipped with gated activation functions (Van Den Oord et al., 2016; Kong et al., 2020). This architecture is cognate with the network employed in (Rasul et al., 2021), albeit with a smaller complement of residual blocks. We recruited Fourier positional embeddings to encode the flow step s into a 32-dimensional vector by means of a fixed set of frequencies, and pressed the selfsame embedding technique into service for the encoding of the time-dependent covariates c . Our models were trained via stochastic gradient descent, using an Adam optimizer with a learning rate of 0.001 and a batch size of 128. Contingent upon the dataset, training to convergence required between 40 and 100 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Baselines We set FLOWTIME against a broad complement of established baselines. These encompass traditional statistical methods, namely Seasonal Naive (SN), AutoARIMA, and AutoETS (Hyndman et al., 2008), alongside deep learning methods such as DeepAR (Salinas et al., 2020) and WaveNet (Van Den Oord et al., 2016). We further incorporate diffusion-based baselines—TSDiff (Kollovich et al., 2023), SSSD (Alcaraz & Strodthoff, 2022), and TimeGrad (Rasul et al., 2021)—together with a flow matching baseline, TSFlow (Kollovich et al., 2024).

Table 2: A test-set CRPS comparison (lower is superior) among the several models on the forecasting of five real-world datasets. The mean and standard-error metrics for FlowTime were obtained by retraining and evaluating five times over; the figures for the baselines are reproduced from (Kollovich et al., 2024). The best scores are rendered in **bold**, the second-best underlined.

Method	Electr.	Exchange	Solar	Traffic	Wikipedia
SN	0.060±0.000	0.013±0.000	0.512±0.000	0.221±0.000	0.423±0.000
ARIMA	0.344±0.000	0.013±0.000	0.558±0.003	0.486±0.000	0.654±0.000
ETS	0.056±0.000	0.008±0.000	0.550±0.000	0.492±0.000	0.651±0.000
DeepAR	0.051±0.000	0.013±0.004	0.429±0.015	0.103±0.000	0.215±0.003
WaveNet	0.058±0.008	0.012±0.001	0.360±0.000	0.099±0.000	0.207 ±0.000
CSDI	0.051±0.000	0.013±0.000	0.360±0.000	0.152±0.000	0.318±0.012
SSSD	0.048±0.000	<u>0.010</u> ±0.000	0.354±0.024	0.107±0.002	<u>0.209</u> ±0.000
TSFlow	<u>0.045</u> ±0.001	0.009 ±0.001	<u>0.343</u> ±0.002	0.083 ±0.000	0.227±0.000
FLOWTIME	0.042 ±0.001	0.009 ±0.001	0.284 ±0.002	<u>0.089</u> ±0.000	0.243±0.002

Results Table 2 sets forth the comparison of our proposed FLOWTIME model against an array of baselines on five real-world timeseries datasets. The results attest that FLOWTIME attains state-of-the-art performance on three of the five datasets—namely, Electricity, Exchange, and Solar. On the Electricity dataset, FLOWTIME surpasses every baseline, registering a CRPS of 0.042 that amounts to a 6.7% improvement over the second-best model, TSFlow (0.045). On the Exchange dataset,

FLOWTIME draws level with TSFlow at a CRPS of 0.009, thereby comfortably eclipsing both the traditional statistical methods and the competing deep learning approaches. Most striking of all, on the Solar dataset FLOWTIME secures a substantial gain, its CRPS of 0.284 constituting a 17.2% reduction in error relative to the next-best model, TSFlow (0.343). Although FLOWTIME occupies the second rank on the Traffic dataset, with a CRPS of 0.089 that trails TSFlow (0.083) by a slender margin, it nonetheless outstrips all remaining traditional and deep learning baselines. On the Wikipedia dataset, FLOWTIME attains a competitive CRPS of 0.243, positioning it behind only WaveNet (0.207) and SSSD (0.209). Considered in the aggregate, these results demonstrate that the autoregressive flow-matching approach embodied in FLOWTIME consistently delivers performance that is superior or, at the least, highly competitive across a diverse spectrum of real-world timeseries forecasting tasks.

5 DISCUSSION

In the present work, we have introduced FLOWTIME, a probabilistic forecasting model that weds autoregressive modeling to flow matching in order to generate accurate and well-calibrated predictions for multivariate timeseries data. By factorizing the forecasting problem into a sequence of conditional distributions, FLOWTIME strikes a felicitous balance between the sequential dependencies inherent in temporal data and the expressive latitude of flow-based generative modeling. Our empirical findings, spanning simulated dynamical systems and real-world datasets alike, demonstrate that FLOWTIME attains performance competitive with state-of-the-art baselines on demanding forecasting benchmarks. The autoregressive decomposition occasions appreciable improvements in the calibration of uncertainty and in distributional coverage, relative to standard flow-matching approaches that model entire future windows in a single, simultaneous pass. Beyond this, FLOWTIME exhibits robust generalization to out-of-distribution scenarios—a capability of cardinal importance for real-world deployment, wherein test conditions not infrequently depart from those encountered during training.

This notwithstanding, FLOWTIME is attended by limitations that merit candid discussion. In the first instance, the sequential character of autoregressive sampling incurs a computational overhead at inference time, by contrast with fully parallelizable non-autoregressive models—a burden that may circumscribe its applicability within real-time forecasting systems. Furthermore, our present implementation applies flow matching directly within the data space, an arrangement that may prove suboptimal for high-dimensional, partially observed data. Prospective avenues of inquiry might profitably explore latent-space formulations, extend the framework to irregularly sampled data, and interrogate broader applications spanning scientific modeling and decision-making under uncertainty. In sum, FLOWTIME constitutes a meaningful advance in probabilistic timeseries forecasting, evincing the promise of flow-based generative models for the concurrent improvement of predictive accuracy and uncertainty quantification.

REFERENCES

- Michael S Albergo, Nicholas M Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *arXiv preprint arXiv:2303.08797*, 2023.
- Juan Miguel Lopez Alcaraz and Nils Strodthoff. Diffusion-based time series imputation and forecasting with structured state space models. *arXiv preprint arXiv:2208.09399*, 2022.
- Alexander Alexandrov, Konstantinos Benidis, Michael Bohlke-Schneider, Valentin Flunkert, Jan Gasthaus, Tim Januschowski, Danielle C Maddix, Syama Rangapuram, David Salinas, Jasper Schulz, et al. GluonTS: Probabilistic and neural time series modeling in python. *Journal of Machine Learning Research*, 21(116):1–6, 2020.
- Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. *Advances in Neural Information Processing Systems*, 31, 2018.
- Fatoumata Dama and Christine Sinoquet. Time series analysis and modeling to forecast: a survey. In *ArXiv:2104.00164*, 2021.
- Abhyuday Desai, Cynthia Freeman, Zuhui Wang, and Ian Beaver. TimeVAE: A variational auto-encoder for multivariate time series generation. *arXiv preprint arXiv:2111.08095*, 2021.

432 Dheeru Dua, Casey Graff, et al. UCI machine learning repository. URL <http://archive.ics.uci.edu/ml>,
433 7(1):62, 2017.

434

435 Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam
436 Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for
437 high-resolution image synthesis. In *Forty-first International Conference on Machine Learning*,
438 2024.

439 Richard FitzHugh. Impulses and physiological states in theoretical models of nerve membrane.
440 *Biophysical Journal*, 1(6):445–466, 1961. doi: 10.1016/S0006-3495(61)86902-6.

441

442 Jan Gasthaus, Konstantinos Benidis, Yuyang Wang, Syama Sundar Rangapuram, David Salinas,
443 Valentin Flunkert, and Tim Januschowski. Probabilistic forecasting with spline quantile function
444 RNNs. In *The 22nd International Conference on Artificial Intelligence and Statistics*, pp. 1901–
445 1910. PMLR, 2019.

446 Will Grathwohl, Ricky TQ Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. FFJORD:
447 Free-Form continuous dynamics for scalable reversible generative models. In *International Con-*
448 *ference on Learning Representations*, 2018.

449

450 Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in*
451 *Neural Information Processing Systems*, 33:6840–6851, 2020.

452 Yang Hu, Xiao Wang, Lirong Wu, Huatian Zhang, Stan Z. Li, Sheng Wang, and Tianlong Chen.
453 Fm-ts: Flow matching for time series generation. In *ArXiv:2411.07506v1 [cs.LG]*, 2024. URL
454 <http://arxiv.org/abs/2411.07506>.

455

456 Rob Hyndman, Anne B Koehler, J Keith Ord, and Ralph D Snyder. *Forecasting with Exponential*
457 *Smoothing: The State Space Approach*. Springer Science & Business Media, 2008.

458

459 Rob J Hyndman and Anne B Koehler. Another look at measures of forecast accuracy. *International*
460 *Journal of Forecasting*, 22(4):679–688, 2006.

461 Maximilian Karl, Maximilian Soelch, Justin Bayer, and Patrick Van der Smagt. Deep varia-
462 tional Bayes filters: Unsupervised learning of state space models from raw data. *arXiv preprint*
463 *arXiv:1605.06432*, 2016.

464

465 Marcel Kollovich, Abdul Fatir Ansari, Michael Bohlke-Schneider, Jasper Zschiegner, Hao Wang,
466 and Yuyang Bernie Wang. Predict, refine, synthesize: Self-guiding diffusion models for proba-
467 bilistic time series forecasting. *Advances in Neural Information Processing Systems*, 36:28341–
468 28364, 2023.

469

470 Marcel Kollovich, Marten Lienen, David Lüdke, Leo Schwinn, and Stephan Günnemann. Flow
471 matching with gaussian process priors for probabilistic time series forecasting. In *The Thirteenth*
472 *International Conference on Learning Representations*, 2024.

473

474 Zhifeng Kong, Wei Ping, Jiaji Huang, Kexin Zhao, and Bryan Catanzaro. DiffWave: A versatile
475 diffusion model for audio synthesis. *arXiv preprint arXiv:2009.09761*, 2020.

476

477 Guokun Lai, Wei-Cheng Chang, Yiming Yang, and Hanxiao Liu. Modeling long- and short-term
478 temporal patterns with deep neural networks. In *The 41st International ACM SIGIR Conference*
479 *on Research & Development in Information Retrieval*, pp. 95–104, 2018.

480

481 R. Lefever and I. Prigogine. Symmetry breaking instabilities in dissipative systems. II. *The Journal*
482 *of Chemical Physics*, 48(4):1695–1700, 1968. doi: 10.1063/1.1668896.

483

484 Bryan Lim and Stefan Zohren. Time-series forecasting with deep learning: A survey. *Philosophical*
485 *Transactions of the Royal Society A*, 379(2194):20200209, 2021.

Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow match-
ing for generative modeling. In *The Eleventh International Conference on Learning Representa-*
tions, 2022.

486 Xingchao Liu, Chengyue Gong, et al. Flow straight and fast: Learning to generate and transfer data
487 with rectified flow. In *The Eleventh International Conference on Learning Representations*, 2022.
488
489 Edward Lorenz. Deterministic nonperiodic flow. *Journal of Atmospheric Sciences*, 20(2), 1963.
490 Alfred J. Lotka. *Elements of Physical Biology*. Williams & Wilkins, 1925.
491 Caspar Meijer and Lydia Y Chen. The rise of diffusion models in time-series forecasting. *arXiv*
492 *preprint arXiv:2401.03006*, 2024.
493
494 Adam Polyak, Amit Zohar, Andrew Brown, Andros Tjandra, Animesh Sinha, Ann Lee, Apoorv
495 Vyas, Bowen Shi, Chih-Yao Ma, Ching-Yao Chuang, et al. MovieGen: A cast of media founda-
496 tion models. *arXiv e-prints*, pp. arXiv–2410, 2024.
497 Kashif Rasul, Abdul-Saboor Sheikh, Ingmar Schuster, Urs Bergmann, and Roland Vollgraf. Mul-
498 tivariate probabilistic time series forecasting via conditioned normalizing flows. *arXiv preprint*
499 *arXiv:2002.06103*, 2020.
500 Kashif Rasul, Calvin Seward, Ingmar Schuster, and Roland Vollgraf. Autoregressive denoising dif-
501 fusion models for multivariate probabilistic time series forecasting. In *International Conference*
502 *on Machine Learning*, pp. 8857–8868. PMLR, 2021.
503
504 David Salinas, Michael Bohlke-Schneider, Laurent Callot, Roberto Medico, and Jan Gasthaus.
505 High-dimensional multivariate forecasting with low-rank gaussian copula processes. *Advances*
506 *in Neural Information Processing Systems*, 32, 2019.
507 David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. DeepAR: Probabilistic
508 forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3):
509 1181–1191, 2020.
510 Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised
511 learning using nonequilibrium thermodynamics. In *International Conference on Machine Learn-*
512 *ing*, pp. 2256–2265. PMLR, 2015.
513 Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben
514 Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint*
515 *arXiv:2011.13456*, 2020.
516
517 Ella Tamir, Najwa Laabid, Markus Heinonen, Vikas Garg, and Arno Solin. Conditional flow match-
518 ing for time series modelling. In *ICML Workshop: Structured Probabilistic Inference and Gener-*
519 *ative Modeling*, 2024.
520 Yusuke Tashiro, Jiaming Song, Yang Song, and Stefano Ermon. CSDI: Conditional score-based
521 diffusion models for probabilistic time series imputation. *Advances in Neural Information Pro-*
522 *cessing Systems*, 34:24804–24816, 2021.
523 Alexander Tong, Kilian Fatras, Nikolay Malkin, Guillaume Hugué, Yanlei Zhang, Jarrod Rector-
524 Brooks, Guy Wolf, and Yoshua Bengio. Improving and generalizing flow-based generative models
525 with minibatch optimal transport. *arXiv preprint arXiv:2302.00482*, 2023.
526
527 Aaron Van Den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves,
528 Nal Kalchbrenner, Andrew Senior, Koray Kavukcuoglu, et al. WaveNet: A generative model for
529 raw audio. *arXiv preprint arXiv:1609.03499*, 12, 2016.
530 B. van der Pol. On ‘relaxation-oscillations’. *The London, Edinburgh, and Dublin Philosophical*
531 *Magazine and Journal of Science*, 2(11):978–992, 1926. doi: 10.1080/14786442608564127.
532 Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent
533 neural networks. *Neural Computation*, 1(2):270–280, 1989.
534 Robert L Winkler, Javier Munoz, José L Cervera, José M Bernardo, Gail Blattenberger, Joseph B
535 Kadane, Dennis V Lindley, Allan H Murphy, Robert M Oliver, and David Ríos-Insua. Scoring
536 rules and the evaluation of probabilities. *Test*, 5:1–60, 1996.
537
538 Jiexia Ye, Weiqi Zhang, Ke Yi, Yongzi Yu, Ziyue Li, Jia Li, and Fugee Tsung. A survey of time
539 series foundation models: Generalizing time series representation with large language model. In
ArXiv:2104.00164, 2024.

A DATASETS

A.1 DYNAMICAL SYSTEMS

We generated both training and testing data for five classical stochastic dynamical systems—namely, the Lorenz, FitzHugh-Nagumo, Lotka-Volterra, Brusselator, and Van der Pol systems—by numerically integrating their respective stochastic differential equations (SDEs). Each such system assumes the general form

$$d\mathbf{x}(t) = \mathbf{f}(\mathbf{x}(t), t; \Theta) dt + \sigma d\mathbf{W}(t),$$

wherein f denotes the deterministic drift, Θ the system parameters, σ a constant diffusion term, and $\mathbf{W}(t)$ a standard Brownian motion. For each system, we sampled 2400 random initial conditions from a uniform distribution over the prescribed range (see Table 3), and for each such initial condition we generated a trajectory by solving the SDE via the Euler-Heun method with a fixed step size. From each trajectory we then sampled 200 equally spaced time points, of which the first 75 were designated as observed data, the succeeding 75 reserved for prediction, and the terminal 50 set aside for extrapolation. Of the 2400 trajectories thus generated, 2000 were apportioned to training and the remaining 400 to testing.

Table 3: A synopsis of the stochastic dynamical systems under consideration. Here $\mathbf{f}(\mathbf{x})$ denotes the drift function, Θ the system parameters, σ the constant diffusion term, and $p(\mathbf{x}(0))$ the distribution over initial conditions. Each system was integrated over the stipulated interval $[t_0, t_1]$ across 200 steps by means of the Euler–Heun method.

System	$f(\mathbf{x})$	Θ	Σ	$p(\mathbf{x}(0))$	$[t_0, t_1]$
Lorenz	$f_1 = \sigma(x_2 - x_1)$	$\sigma = 10$	(1.5, 1.5, 1.5)	$\mathcal{U}([0, 10])$	[0, 2]
	$f_2 = x_1(\rho - x_3) - x_2$	$\rho = 28$			
	$f_3 = x_1 x_2 - \beta x_3$	$\beta = \frac{8}{3}$			
FitzHugh–Nagumo	$f_1 = x_1 - \frac{x_1^3}{3} - x_2 + I$	$a = 0.7$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 10]
	$f_2 = \frac{x_1 + a - b x_2}{\tau}$	$b = 0.8$			
		$\tau = 12.5$			
Lotka–Volterra	$f_1 = \alpha x_1 - \beta x_1 x_2$	$\alpha = 1.3$	(1.5, 1.5)	$\mathcal{U}([0, 5])$	[0, 20]
	$f_2 = -\delta x_2 + \gamma x_1 x_2$	$\beta = 0.9$			
		$\gamma = 0.8$			
Brusselator	$f_1 = A + x_1^2 x_2 - (B + 1) x_1$	$\delta = 1.8$	(1.5, 1.5)	$\mathcal{U}([0, 2])$	[0, 20]
	$f_2 = B x_1 - x_1^2 x_2$	$A = 1.0$			
		$B = 3.0$			
Van der Pol	$f_1 = x_2$	$\mu = 0.1$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 20]
	$f_2 = \mu(1 - x_1^2) x_2 - x_1$				

A.2 REAL-WORLD DATASETS

We availed ourselves of five distinct datasets from GluonTS (Alexandrov et al., 2020) in order to evaluate our model’s performance; in each case we adopted the versions preprocessed after the fashion of (Salinas et al., 2019). A synopsis of their properties is enumerated in Table 4.

Table 4: The salient properties of the datasets marshaled in our experiments: the dimension n , the domain Ω , the frequency, the aggregate number of training timesteps, and the prediction length.

Dataset	n	Ω	freq.	timesteps	prediction length
Electricity	370	$\mathbb{R}^+$	hourly	5833	24
Exchange	8	$\mathbb{R}^+$	daily	6071	30
Solar	137	$\mathbb{R}^+$	hourly	7009	24
Traffic	963	(0, 1)	hourly	4001	24
Wikipedia	2000	$\mathbb{N}$	daily	792	30

B NON-AUTOREGRESSIVE FLOW MATCHING BASELINE

Problem Setting We consider the same forecasting setup as that of Section 3, wherein, given past observations $\mathbf{Y}_l$ and covariates $\mathbf{C}$, our aim remains the modeling of the conditional distribution $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$. In contradistinction to the autoregressive factorization, however, we here model directly the joint distribution over the entire future trajectory $\mathbf{Y}_f \in \mathbb{R}^{f \times n}$, forgoing any temporal factorization:

$$p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C}) = p(\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}|\mathbf{Y}_l, \mathbf{C}) \quad (9)$$

This formulation preserves the temporal correlations obtaining across all future time steps, yet does so at the cost of having to learn a high-dimensional distribution.

Training We construct a probability path $(p^s(\mathbf{Y}_f))^{0 \leq s \leq 1}$ that transports samples from a prior distribution $p^0(\mathbf{Y}_f) = \mathcal{N}(0, \Sigma)$ onto the target distribution $p^1(\mathbf{Y}_f) \approx p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$, wherein Σ constitutes a block-diagonal covariance matrix. This prior admits interpretation as a collection of independent, per-dimension Brownian motion processes, a Brownian motion being understood here as a stochastic process $W: [\tau+1, \tau+f] \rightarrow \mathbb{R}^n$.

We define the conditional probability path by means of a linear interpolation bridge perturbed by Brownian noise:

$$p^s(\mathbf{Y}_f|\mathbf{z}) = \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma) \quad (10)$$

where $\mathbf{z} = (\mathbf{Y}_f^0, \mathbf{Y}_f^1)$ with $\mathbf{Y}_f^0 \sim p^0$ and $\mathbf{Y}_f^1 \sim p^1$. The corresponding conditional velocity field then assumes the form:

$$\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0 + \frac{\sigma^2(1-2s)}{2}\Sigma^{-1}(\mathbf{Y}_f - ((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1)) \quad (11)$$

In the limit $\sigma^2 \rightarrow 0$, this collapses to the rectilinear-path velocity $\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0$. We learn a neural velocity field ν_θ that acts upon the entire future trajectory. The context encoding remains cognate with that of Eq. (7):

$$\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f}) \quad (12)$$

where ζ_ϕ now ingests both past and future covariates alike. The training objective thereupon becomes:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{Y}_f^s \sim p^s(\mathbf{Y}_f|\mathbf{z})} \|\mu(\mathbf{Y}_f^s, s|\mathbf{z}) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|^2 \quad (13)$$

where ν_θ is realized as a sequential neural network (for instance, a Transformer or an RNN) that processes the trajectory in its entirety. Sampling, in turn, necessitates solving the trajectory-level ODE:

$$\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s), \quad \psi(\mathbf{Y}_f, 0) \sim p^0 \quad (14)$$

by recourse to numerical solvers. This procedure yields joint samples from $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$ without appeal to any autoregressive decomposition.

Implementation Details We enlisted this baseline to conduct experiments on the task of forecasting multiple dynamical systems. For this configuration, we instantiated ν_θ as a 4-layer bi-directional LSTM endowed with 128 hidden units, retaining the same training parameters as those set out in Section 4.1. Training to convergence required approximately 2 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Key Differences from Autoregressive Variant The non-autoregressive approach diverges in fundamental respects from its autoregressive counterpart embodied in FLOWTIME. In the first place, whereas the autoregressive method factorizes the joint distribution through a Markovian decomposition across time steps, the non-autoregressive baseline models the full future trajectory $\mathbf{Y}_f$ directly, treating it as a single high-dimensional random variable; this expedient preserves cross-temporal dependencies at the price of learning a more intricate distribution over $\mathbb{R}^{n \times f}$. In the second place, the two diverge appreciably in their computational profiles: the non-autoregressive variant propagates entire trajectories through sequential neural networks in estimating the velocity fields, thereby enabling the parallel generation of all future timepoints, albeit at the cost of heightened memory demands. Finally, their respective mechanisms of uncertainty propagation stand in sharp contrast: the

Algorithm 3 Non-Autoregressive Flow Matching: Training

Require: Dataset $\mathcal{D}$, networks ν_θ, ζ_ϕ

- 1: **while** not converged **do**
- 2: Sample $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$ and $(\mathbf{Y}_f^1, \mathbf{Y}_l, \mathbf{C}) \sim \mathcal{D}$
- 3: Set $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f})$
- 4: Sample $s \sim \mathcal{U}(0, 1)$
- 5: $\mathbf{Y}_f^s \sim \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma)$
- 6: Compute target $\mu(\mathbf{Y}_f^s, s|\mathbf{z})$
- 7: Update θ, ϕ to minimize $\|\mu(\cdot) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|_{\Sigma^{-1}}^2$
- 8: **end while**

Algorithm 4 Non-Autoregressive Flow Matching: Sampling

Require: $\mathbf{Y}_l, \mathbf{C}$, networks ν_θ, ζ_ϕ

- 1: Encode context: $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C})$
- 2: Sample initial trajectory $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$
- 3: Solve $\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)$
- 4: Return $\psi(\mathbf{Y}_f, 1)$

autoregressive approach models explicitly the manner in which prediction errors accumulate along the chain of conditional distributions, whereas the non-autoregressive method apprehends the joint uncertainty over all timesteps through trajectory-level sampling, yet is bereft of any explicit apparatus for modeling the dynamics of error accumulation. These distinctions give rise to complementary trade-offs: the autoregressive variant affords interpretable uncertainty quantification together with efficient, window-based computation, though at the price of positing only limited Markovian dependencies, while the non-autoregressive baseline preserves the full temporal correlations at a greater computational expense and with a less explicit account of uncertainty dynamics.

C EVALUATION METRICS

In this section, we expound the metrics enlisted to appraise the quality of our forecasts. Given an n -dimensional time series $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$ together with its predictive distribution—or, equivalently, samples drawn from our probabilistic model—we avail ourselves of two metrics: the Continuous Ranked Probability Score (CRPS) and the Normalized Root Mean Squared Error (NRMSE).

C.1 CONTINUOUS RANKED PROBABILITY SCORE (CRPS)

The CRPS is a proper scoring rule widely enlisted to adjudge the quality of probabilistic forecasts. For a univariate random variable X possessed of a cumulative distribution function (CDF) F_X , together with an observed value x , the CRPS is defined as:

$$\text{CRPS}(F_X, x) = \int_{-\infty}^{+\infty} (F_X(y) - \mathbf{1}\{y \geq x\})^2 dy, \quad (15)$$

where $\mathbf{1}\{\cdot\}$ denotes the indicator function. Intuitively, the CRPS quantifies the fidelity with which the entire predictive distribution accords with the observation x , so that lower CRPS values betoken predictive distributions that are at once better calibrated and more accurate.

Within the context of time series forecasting, we may compute the CRPS at each time step of the forecasting horizon and thereafter average the resulting values:

$$\text{CRPS}_{\text{avg}} = \frac{1}{f} \sum_{k=1}^f \text{CRPS}(F_{X_{\tau+k}}, y_{\tau+k}), \quad (16)$$

where $F_{X_{\tau+k}}$ denotes the predicted CDF (or an empirical CDF derived from samples) at forecast horizon k , and $y_{\tau+k}$ the corresponding ground-truth observation.

C.2 NORMALIZED ROOT MEAN SQUARED ERROR (NRMSE)

The Root Mean Squared Error (RMSE) constitutes a standard metric for appraising the accuracy of point forecasts. For a set of predictions $\{\hat{y}_{\tau+1}, \dots, \hat{y}_{\tau+f}\}$ together with the corresponding ground-truth values $\{y_{\tau+1}, \dots, y_{\tau+f}\}$, the RMSE is given by:

$$\text{RMSE} = \sqrt{\frac{1}{f} \sum_{k=1}^f (\hat{y}_{\tau+k} - y_{\tau+k})^2}. \quad (17)$$

In order to render the RMSE scale-invariant and thereby to facilitate comparison across disparate datasets or time series of differing magnitudes, we employ the Normalized RMSE (NRMSE). One prevalent normalization proceeds by the standard deviation of the ground-truth series, σ_y :

$$\text{NRMSE} = \frac{\text{RMSE}}{\sigma_y}, \quad (18)$$

where σ_y denotes the sample standard deviation of the observations. Alternatively, one might, contingent upon the application at hand, normalize instead by the range of the data ($\max(y) - \min(y)$). A lower NRMSE, in either case, betokens predictions that are more accurate relative to the intrinsic variability of the time series.